

Assessment Solutions

Complete set: 4 coding/prompt questions + 20 MCQs.

Question 1 — Duplicate Message Detection (Java)

Problem: A messaging app prevents duplicate messages. If a message with the same text was delivered in the last k seconds, the new one is dropped. Given timestamps, messages, and k , return `"true"` for delivered and `"false"` for dropped.

Function: `getMessageStatus(List<Integer> timestamps, List<String> messages, int k)`

Example: `timestamps=[1,4,5,9,10,14]`, `messages=["hello","bye","bye","bye","bye","hello"]`, `k=5` →
`["true","true","false","false","true","true"]`

Answer

```
java
```

```

import java.io.*;
import java.util.*;

class Result {
    public static List<String> getMessageStatus(List<Integer> timestamps, List<String> messages, int k) {
        List<String> result = new ArrayList<>();
        Map<String, Integer> lastDelivered = new HashMap<>();

        int n = timestamps.size();

        for (int i = 0; i < n; i++) {
            String msg = messages.get(i);
            int time = timestamps.get(i);

            if (lastDelivered.containsKey(msg) && time - lastDelivered.get(msg) <= k) {
                result.add("false");
            } else {
                result.add("true");
                lastDelivered.put(msg, time);
            }
        }

        return result;
    }
}

```

```

public class Solution {
    public static void main(String[] args) throws IOException {
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.out));

        int timestampsCount = Integer.parseInt(bufferedReader.readLine().trim());
        List<Integer> timestamps = new ArrayList<>();
        for (int i = 0; i < timestampsCount; i++) {
            int item = Integer.parseInt(bufferedReader.readLine().trim());
            timestamps.add(item);
        }

        int messagesCount = Integer.parseInt(bufferedReader.readLine().trim());
        List<String> messages = new ArrayList<>();
        for (int i = 0; i < messagesCount; i++) {
            messages.add(bufferedReader.readLine());
        }
    }
}

```

```

int k = Integer.parseInt(bufferedReader.readLine().trim());

List<String> result = Result.getMessageStatus(timestamps, messages, k);
for (int i = 0; i < result.size(); i++) {
    bufferedWriter.write(result.get(i));
    if (i != result.size() - 1) bufferedWriter.newLine();
}
bufferedWriter.newLine();
bufferedWriter.close();
bufferedReader.close();
}
}

```

Logic: Keep a `HashMap<message, lastDeliveredTime>`. For each incoming message, if `currentTime - lastTime <= k`, drop it (don't update the map). Otherwise deliver it and update the map.

Question 2 — Address Management (React, MERN E-commerce)

Task: Build an address management section in the user dashboard. Save/edit/delete addresses with localStorage. During checkout, show saved addresses in a dropdown; new typed addresses get auto-saved.

Answer — `frontend/src/components/layout/User/AddressManager.js`

jsx

```
import React, { useState, useEffect } from "react";
import UserMenu from "../UserMenu";
import "../AddressManager.css";

const STORAGE_KEY = "savedAddresses";

function AddressManager() {

  const [addresses, setAddresses] = useState([]);
  const [showForm, setShowForm] = useState(false);
  const [editIndex, setEditIndex] = useState(-1);
  const [form, setForm] = useState({ street: "", city: "", state: "", zipCode: "" });
  const [errors, setErrors] = useState({});

  useEffect(() => {
    const stored = localStorage.getItem(STORAGE_KEY);
    if (stored) {
      try {
        const parsed = JSON.parse(stored);
        if (Array.isArray(parsed)) setAddresses(parsed);
        else setAddresses([]);
      } catch (e) {
        setAddresses([]);
      }
    }
  }, []);

  const persist = (list) => {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(list));
    setAddresses(list);
  };

  const handleInputChange = (e) => {
    setForm({ ...form, [e.target.name]: e.target.value });
  };

  const validateForm = () => {
    const errs = {};
    if (!form.street.trim()) errs.street = "Street is required";
    if (!form.city.trim()) errs.city = "City is required";
    if (!form.state.trim()) errs.state = "State is required";
    if (!form.zipCode.trim()) errs.zipCode = "ZIP Code is required";
    return errs;
  };
}
```

```

};

const handleAddClick = () => {
  setForm({ street: "", city: "", state: "", zipCode: "" });
  setErrors({});
  setEditIndex(-1);
  setShowForm(true);
};

const handleEdit = (i) => {
  setForm(addresses[i]);
  setEditIndex(i);
  setErrors({});
  setShowForm(true);
};

const handleDelete = (i) => {
  const updated = addresses.filter((_, idx) => idx !== i);
  persist(updated);
};

const handleSubmit = (e) => {
  e.preventDefault();
  const validationErrors = validateForm();
  if (Object.keys(validationErrors).length > 0) {
    setErrors(validationErrors);
    return;
  }

  let updated;
  if (editIndex >= 0) {
    updated = addresses.map((a, i) => (i === editIndex ? form : a));
  } else {
    updated = [...addresses, form];
  }
  persist(updated);

  setForm({ street: "", city: "", state: "", zipCode: "" });
  setErrors({});
  setEditIndex(-1);
  setShowForm(false);
};

return (

```

```

<div className="container-fluid p-3 m-3">
  <div className="row">
    <div className="col-md-3">
      <UserMenu />
    </div>

    <div className="col-md-9">
      <div className="address-manager-container">

        {!showForm && (
          <div>
            <h3 className="save-address-heading">Saved Addresses</h3>

            {addresses.length === 0 ? (
              <p data-testid="no-addr" className="no-saved-addr">
                No saved address yet.
              </p>
            ) : (
              <ul className="address-list">
                {addresses.map((addr, i) => (
                  <li key={i} className="address-item">
                    <p data-testid="addr-street">Street: {addr.street}</p>
                    <p data-testid="addr-city">City: {addr.city}</p>
                    <p data-testid="addr-state">State: {addr.state}</p>
                    <p data-testid="addr-zipCode">ZIP Code: {addr.zipCode}</p>
                    <button data-testid="addr-edit" className="save-address-btn">
                    <button data-testid="addr-delete" className="save-address-bt
                  </li>
                ))}
              </ul>
            )}

            <button data-testid="addr-btn" className="btn btn-primary address-fc
              Add New Address
            </button>
          </div>
        )}

        {showForm && (
          <div>
            <h2 className="manage-address-heading">Manage Address</h2>

            <form className="address-form" onSubmit={handleSubmit}>
              <div className="mb-3">

```

```

        <label htmlFor="street" className="form-label">Street:</label>
        <input type="text" className="form-control" id="street" name="street"
        {errors.street && <span className="text-danger">{errors.street}</span>}
    </div>
    <div className="mb-3">
        <label htmlFor="city" className="form-label">City:</label>
        <input type="text" className="form-control" id="city" name="city"
        {errors.city && <span className="text-danger">{errors.city}</span>}
    </div>
    <div className="mb-3">
        <label htmlFor="state" className="form-label">State:</label>
        <input type="text" className="form-control" id="state" name="state"
        {errors.state && <span className="text-danger">{errors.state}</span>}
    </div>
    <div className="mb-3">
        <label htmlFor="zipCode" className="form-label">ZIP Code:</label>
        <input type="text" className="form-control" id="zipCode" name="zipCode"
        {errors.zipCode && <span className="text-danger">{errors.zipCode}</span>}
    </div>
    <button type="submit" className="btn btn-primary">
        {editIndex >= 0 ? "Update Address" : "Save Address"}
    </button>
</form>
</div>
    )}

    </div>
  </div>
</div>
</div>
);
}

export default AddressManager;

```

Answer — frontend/src/components/views/checkout/Checkout.jsx

jsx

```
import React, { useState, useEffect } from "react";
import { useNavigate } from "react-router-dom";
import QuotaCheck from "../../layout/User/QuotaCheck.jsx";
import "./checkout.css";

const STORAGE_KEY = "savedAddresses";

function Checkout() {
  const navigate = useNavigate();

  const [address, setAddress] = useState({ street: "", city: "", state: "", zipCode: "" });
  const [savedAddresses, setSavedAddresses] = useState([]);
  const [selectedIndex, setSelectedIndex] = useState("");
  const [errors, setErrors] = useState({});
  const [quotaValid, setQuotaValid] = useState(true);
  const [quotaError, setQuotaError] = useState(null);

  useEffect(() => {
    const stored = localStorage.getItem(STORAGE_KEY);
    if (stored) {
      try {
        const parsed = JSON.parse(stored);
        if (Array.isArray(parsed)) setSavedAddresses(parsed);
      } catch (e) {
        setSavedAddresses([]);
      }
    }
  }, []);

  const handleInputChange = (e) => {
    setAddress({ ...address, [e.target.name]: e.target.value });
    setSelectedIndex("");
  };

  const handleSelectChange = (e) => {
    const idx = e.target.value;
    setSelectedIndex(idx);
    if (idx === "") setAddress({ street: "", city: "", state: "", zipCode: "" });
    else setAddress(savedAddresses[idx]);
  };

  const handleQuotaCheck = (isValid, error) => {
    setQuotaValid(isValid);
  };
}
```

```

    setQuotaError(error);
  };

const validateForm = () => {
  const errs = {};
  if (!address.street.trim()) errs.street = "Street is required";
  if (!address.city.trim()) errs.city = "City is required";
  if (!address.state.trim()) errs.state = "State is required";
  if (!address.zipCode.trim()) errs.zipCode = "ZIP code is required";
  return errs;
};

const handleSubmit = (e) => {
  e.preventDefault();
  const validationErrors = validateForm();
  if (Object.keys(validationErrors).length === 0 && quotaValid) {
    if (selectedIndex === "") {
      const exists = savedAddresses.some(
        (a) => a.street === address.street && a.city === address.city &&
          a.state === address.state && a.zipCode === address.zipCode
      );
      if (!exists) {
        const updated = [...savedAddresses, address];
        localStorage.setItem(STORAGE_KEY, JSON.stringify(updated));
      }
    }
    navigate("/checkout/payment");
  } else {
    setErrors(validationErrors);
  }
};

return (
  <div className="checkout-container">
    <QuotaCheck onQuotaCheck={handleQuotaCheck} />
    <form onSubmit={handleSubmit} className="checkout-form">
      <h2>Shipping Address</h2>

      <div className="mb-3">
        <label htmlFor="street" className="form-label">Street:</label>
        <input type="text" data-testid="checkout-street" className="form-control"
          {errors.street && <span className="text-danger">{errors.street}</span>}
        </div>

```

```

<div className="mb-3">
  <label htmlFor="city" className="form-label">City:</label>
  <input type="text" data-testid="checkout-city" className="form-control" id=
    {errors.city && <span className="text-danger">{errors.city}</span>}
</div>

<div className="mb-3">
  <label htmlFor="state" className="form-label">State:</label>
  <input type="text" data-testid="checkout-state" className="form-control" id=
    {errors.state && <span className="text-danger">{errors.state}</span>}
</div>

<div className="mb-3">
  <label htmlFor="zipCode" className="form-label">ZIP Code:</label>
  <input type="text" data-testid="checkout-zipCode" className="form-control" id=
    {errors.zipCode && <span className="text-danger">{errors.zipCode}</span>}
</div>

<button type="submit" className="btn btn-primary" disabled={!quotaValid}>
  {quotaValid ? "Checkout" : "Order Limits Exceeded"}
</button>

<div className="mb-3 custom-select">
  <label htmlFor="addressSelect" className="form-label saveAddr-label">
    Have you already saved an address? View the options in the dropdown menu
  </label>
  <select id="addressSelect" className="form-control" data-testid="addr-select">
    <option value="">New Address</option>
    {savedAddresses.map((a, i) => (
      <option key={i} value={i}>
        {a.street}, {a.city}, {a.state} - {a.zipCode}
      </option>
    ))}
  </select>
</div>
</form>
</div>
);
}

export default Checkout;

```

Question 3 — RAG Pipeline (Python, FastEmbed)

Task: Build the data prep + indexing part of a RAG pipeline. Load `.txt` content, preprocess (lowercase), embed via FastEmbed, store in in-memory list.

Answer — `document_loader.py`

```
python
```

```
from typing import List

class DocumentLoader:
    """Loads and preprocesses documents from a file."""

    def __init__(self, filepath: str):
        self.filepath = filepath

    def load_documents(self) -> List[str]:
        """Read the file and return a list of non-empty paragraph strings."""
        with open(self.filepath, "r", encoding="utf-8") as f:
            content = f.read()
            paragraphs = [p.strip() for p in content.split("\n\n")]
            paragraphs = [p for p in paragraphs if p]
            return paragraphs

    def preprocess_documents(self, docs: List[str]) -> List[str]:
        """Lowercase each document string and return the processed list."""
        return [doc.lower() for doc in docs]
```

Answer — `vector_store.py`

```
python
```

```

from fastembed import TextEmbedding
from typing import List

class Document:
    def __init__(self, page_content: str):
        self.page_content = page_content

class VectorStoreBuilder:
    def __init__(self):
        self.model = TextEmbedding(model_name="BAAI/bge-small-en-v1.5")
        self.vectorstore = []

    def build_vectorstore(self, docs: List[str]):
        embeddings = list(self.model.embed(docs))
        for text, vector in zip(docs, embeddings):
            self.vectorstore.append((vector, Document(text)))

```

Key point: `TextEmbedding(model_name="BAAI/bge-small-en-v1.5")` must pass `model_name` explicitly — the test uses `assert_called_once_with(model_name=...)`.

Question 4 — RELEVANT / IRRELEVANT Classifier Prompt

Task: Write a prompt that reads a Question and Answer pair and outputs either `RELEVANT` or `IRRELEVANT`.

Answer

```

Classify whether the Answer is on-topic for the Question.

Output RELEVANT if the answer addresses the same subject as the question, even if
incomplete or imprecise.
Output IRRELEVANT if the answer discusses a different subject, even if it shares
some keywords.

Do not judge accuracy or quality – only whether the answer is on-topic.

Output exactly one word: RELEVANT or IRRELEVANT. No punctuation, no explanation.

```

Question: {{question}}

Answer: {{answer}}

MCQs — Questions 5 to 24

#	Question	Answer
5	Ephemeral storage	Temporary storage that is lost when the instance is stopped or terminated
6	Encryption at Rest	Protecting data while it is stored on a disk or in a database
7	Reinforcement Learning based on	An agent receiving rewards or penalties based on its actions
8	Dropout in neural network	Randomly setting a fraction of input units to zero at each update during training time
9	GRU (Gated Recurrent Unit)	A simplified version of LSTM with fewer gates
10	Global Minimum in optimization	The point where the loss function has the lowest possible value
11	Why Deep Learning for unstructured data	It can automatically learn feature representations from raw data
12	Why Agent more complex than single Tool-LLM call	Because it involves a dynamic reasoning loop and state maintenance
13	Tool-Selection	The LLM choosing the most appropriate tool from its available toolkit
14	Human-in-the-loop (HITL)	A design where a human reviews or approves an agent's actions
15	Multi-Agent System (MAS)	A system where multiple agents collaborate or compete to solve tasks
16	Deep Learning vs traditional ML	Automatically learn features using multiple layers
17	Generative AI capability	Producing new content based on learned patterns
18	Most suitable Generative AI task	Generating a summary from text

#	Question	Answer
19	Why API Gateway over direct service calls	It simplifies client integration by hiding internal service topology
20	Single responsibility in microservices	Each service focuses on one business capability
21	Reduces tight coupling between services	Well-defined APIs/contracts between services
22	Best prompt for deterministic output	Specify steps and required format explicitly
23	Context window in LLMs	The maximum amount of tokens the model can attend to at once
24	Performs well on training, poorly on new data	Overfitting

That's the whole set — 4 coding/prompt questions + 20 MCQs.